

SOVEREIGN · GOVERNED · SELF-HOSTABLE

Sovereign Agentic OS

The product guide — install it, run it, and put every
governed workflow to work.

Orchestrated by Data Masterclass • datamasterclass.com • www.sovereign-agentic.com

Chart 0.2.10 (app 0.2.0-alpha.11) • generated 2026-07-02 from commit 410a249

Contents

1 Welcome

- 1.1 What the Sovereign Agentic OS is
 - 1.2 Who it is for
 - 1.3 Three principles
 - 1.4 How the platform is layered
-

2 The operating model

- 2.1 Everything is a governed artifact
 - 2.2 The promotion ladder
 - 2.3 Four roles, assigned per domain
 - 2.4 The governance spine
 - 2.5 Every assistant acts, not just chats
-

3 Getting started

- 3.1 What you need
 - 3.2 Install in one command
 - 3.3 Open the front door
 - 3.4 Bootstrap: the first administrator, then real users and roles
 - 3.5 Try the seeded demos
-

4 Using the platform, tab by tab

- 4.1 Home – the golden-path launcher
- 4.2 Cockpit – what's moving, what needs you
- 4.3 Strategy – pillars, value and adoption
- 4.4 Big Bets – initiative roadmaps
- 4.5 Agents – compose, govern, run
- 4.6 Software – build governed apps, sovereign
- 4.7 Science – classic ML (opt-in, Layer 4)
- 4.8 Knowledge – the domain's operating manual
- 4.9 Files – a calm governed drive
- 4.10 Data – datasets, refined and governed

- 4.11 Metrics – one number, everywhere
 - 4.12 Dashboards – governed BI
 - 4.13 Connections – governed bridges to outside systems
 - 4.14 Marketplace – consume across domains
 - 4.15 Monitoring – artifact observability
 - 4.16 Governance – the control plane
 - 4.17 The Platform section
 - 4.18 Tutorials – learn any path in place
-

5 Reference

- 5.1 Deploying to your cloud (STACKIT)
 - 5.2 Sizing & capacity
 - 5.3 Durable lakehouse
 - 5.4 Security model
 - 5.5 Models & the LLM gateway
 - 5.6 Use the OS from Claude or ChatGPT (MCP)
 - 5.7 Memory and the build-and-toggle defaults
 - 5.8 Components at a glance
 - 5.9 Full demo-login table (profile `local`)
 - 5.10 Troubleshooting
 - 5.11 Version & changelog
-

1 Welcome

The Sovereign Agentic OS is orchestrated by Data Masterclass. Project home and downloads: www.sovereign-agentic.com.

This is the guide for the people who use the platform – administrators standing it up, builders governing it, and the teams who create and consume data, knowledge, agents and software inside it. It is written to be read top to bottom once, then kept beside you as a reference.

It is in three parts:

1. **Understand it** – what the Sovereign Agentic OS is, who it is for, and the operating model every part of the product shares.
2. **Get started** – install the platform, open it, and bootstrap your first administrator, real users, and roles.
3. **Use it, tab by tab** – one short chapter per tab, in **sidebar order**, each built around the **golden path**: the exact steps a real person follows in the screen, who plays which role, and how that tab hands off to the others.

A closing **Reference** covers cloud deployment, the security model, component details, and the demo logins.

1.1 What the Sovereign Agentic OS is

The **Sovereign Agentic OS** is a self-hostable, EU-residency platform that assembles roughly two dozen best-in-class, permissively-licensed open-source tools into a single **governed** stack – one place where every business **domain** can create, store, use, document and share data, knowledge, dashboards, agents and software.

It ships as **one umbrella Helm chart** (`charts/sovereign-agentic-os`) that brings the whole platform up on any Kubernetes cluster. The default install is **fully self-contained**: every backend runs inside the chart and a small local model stands in for a cloud LLM, so nothing external – and no API key – is required to see the system work end to end. The same chart scales from a laptop to a sovereign production deployment on **STACKIT** (the EU/Germany cloud); the only difference is a values choice – `mode: bundled | external` – per backend.

1.2 Who it is for

- **Regulated organizations, the public sector, and EU enterprises** that need data residency, full audit trails, and zero dependency on US-controlled cloud or hosted LLM platforms.
- **Teams that cannot send data to hosted AI services** but still want production-grade agentic workflows – RAG agents, a lakehouse, BI, and software delivery – all self-hosted.
- **Data Masterclass participants** building real agentic systems on the *same* production components used in the field, not teaching forks.

1.3 Three principles

1. **Permissive open source only.** Every bundled component is Apache-2.0 / MIT / BSD / PostgreSQL licensed – full code auditability, no proprietary lock-in, the right to host and modify indefinitely. This drove deliberate choices: **Valkey** instead of Redis (relicensed), **OpenSearch** for vector *and* lexical retrieval (no pgvector), **Forgejo** instead of Gitea, **Langfuse** instead of LangSmith.
2. **Security and governance are the operating model, not an add-on.** The platform is *secure by default*: agents have no raw internet, every tool call is authorized, every model call is metered and traced, and no real secret ever lives in git.
3. **Self-hosted, in your region.** The production target is STACKIT Kubernetes with STACKIT Object Storage in **EU01 / Deutschland Süd**; model calls can route to **STACKIT AI Model Serving** so prompts and completions never leave the sovereign boundary.

1.4 How the platform is layered

You can reason about – and enable – the platform in layers:

- **Layer 1 – Agent core.** The runtime: **LangGraph** agents calling **LiteLLM** (the one model + tool gateway, with per-key access control and cost caps), every action traced in **Langfuse**, retrieving over **OpenSearch** (hybrid vector + lexical).
- **Layer 2 – Foundations.** Turning raw data and knowledge into governed products: **OPA** (policy-as-code at the tool boundary), **Docling** (document parsing), **Haystack** (RAG), **Dagster** (orchestration), **dbt** (transforms), **Cube** (the metrics layer), **OpenMetadata** (catalog + lineage).
- **Layer 3 – Self-service.** Query, visualize, and ship: the **Iceberg** lakehouse with **central Trino** as the one governed query engine, **Superset** for dashboards, and **in-cluster Forgejo + Argo CD** for software delivery (git → CI → GitOps).
- **Layer 4 – Science / ML.** Classic ML – **JupyterHub**, **MLflow**, **Featureform**, **KServe** – *opt-in and off by default* (heavier, GPU-oriented).
- **Security baseline** spans every layer: default-deny egress through a single proxy chokepoint, a governed `web_fetch`, OPA tool authorization, externalized secrets, hardened pods.

On top sits the **front door** you open in a browser: the **OS UI** (for everyone), with the stack's operational console embedded inside it under **Platform → Components** – there is no separate admin service to run.

Where this release stands. Layers 1–3 are in place; **Science (Layer 4) is opt-in / off by default**. The **OS UI is v1.0** – every sidebar tab is a real surface, brand-themed, with a light/dark theme (light is the default). Most flows run **live against the cluster** and fall back to an honest, clearly-labelled offline mock on a local `kind` cluster. A few areas are explicitly v1 scaffolds – most notably some **Connections** create/build flows (see that chapter). The core is **Apache-2.0**; bundled components keep their own licenses.

2 The operating model

Before the tabs, one idea ties them together. Learn it once and every screen reads the same way.

2.1 Everything is a governed artifact

Whatever you make – a data product, a knowledge workflow, a file, an agent, an app, an ML model, a metric, a connection, a dashboard – it is an **artifact** with the same four attributes: **owner • domain • type • visibility**. And whatever the type, it travels the same lifecycle:

Create → Document → Use → Promote – authored in the OS UI, which scaffolds the real tool underneath (a dbt model, a Cube metric, a Forgejo repo, a KServe service), preview-first, cataloged and audited.

2.2 The promotion ladder

Visibility widens one rung at a time, and each rung is **role-gated**:

VISIBILITY	MEANING	WHO CAN PROMOTE
Personal	the creator only – the default for drafts and app-created data	–
Shared (domain)	usable across the owning domain	Builder or Administrator
Marketplace (certified)	discoverable and importable by other domains	Administrator only

Nothing enters the governed store without **documentation + passing checks** – a transparency gate that turns green only when an artifact is documented and in the lineage graph. Throughout the product the creator-only scope is always labelled "**Personal**" (never "Mine" or "My"), and a domain is named directly – "**domain**", not "My Domain".

2.3 Four roles, assigned per domain

ROLE	WHAT THEY DO
User	consumes – reads dashboards, asks agents, imports certified products
Creator	builds drafts – datasets, workflows, agents, apps (Personal by default)
Builder	certifies and shares in-domain; approves go-lives, deploys and write-backs
Administrator	sets tenant guardrails; promotes to the Marketplace; runs Admin (Platform)

Roles are assigned **per domain** and **compiled to OPA**, so a role change takes effect everywhere at once. Identities are backed by **Ory** – the UI never handles a raw password.

2.4 The governance spine

One **gateway** (every model and tool call, with cost caps), one **policy engine** (OPA, deciding `allow` / `deny` / `requires_approval`), one **trace** (Langfuse), one **audit**. Two layers of policy stack: **tenant guardrails** that Administrators set and domains cannot override (default-deny egress, no plaintext secrets to agents, no cross-domain data without a grant, a model allowlist), and **domain policy** that Builders set within them. High-stakes actions don't fail silently – they queue as a **card** in the **Governance** inbox, where *approving is the action*: on approve, the platform executes the governed effect and writes the audit.

Three planes stay deliberately separate, and cross-link rather than duplicate:

- **Admin** (the "Platform Admin" control room, labelled **Admin** in the sidebar's Platform section) – *configures* the tenant (identity, models, egress, structure).
- **Governance** – *decides and records* (approvals, policy, audit, caps, access).
- **Monitoring** – *observes the artifacts* (runs, spend, traces, drift). **Components** watches the infrastructure (service + cluster health). And **Dashboards** watch the business KPIs.

2.5 Every assistant acts, not just chats

Each tab's built-in assistant is **agentic**, not a chat box. It runs one loop: **PLAN** with the reasoning tier, then **ACT** in a bounded tool-calling loop with the execution tier – calling that tab's governed tools, which are the *same* OPA-authorized, Langfuse-traced functions the UI uses (never a privileged path) – then **verifies**, and for anything side-effectful **stops at a human gate**. The Software build assistant is the fullest example: it scaffolds a repo, writes and commits code, previews the app, and assembles a **deploy review card** through `lib/software` – it never self-approves a go-live. Which models fill the two phases is set once at the gateway (see *Models & the LLM gateway*).

3 Getting started

3.1 What you need

- A container runtime + the `docker` CLI (Docker Desktop or Colima), **running**.
- `kind`, `helm`, and `kubectl` on your `PATH`.
- About **14 GB RAM / 6 CPU** free for the runtime VM (the slice is RAM-bound).

3.2 Install in one command

```
# prereqs: docker (running), kind, helm, kubectl
./install.sh           # press Enter through every prompt
```

Pressing **Enter** through every prompt gives the **fully self-contained** install: every backend is bundled and a small local model answers model calls – nothing external is required. `install.sh` creates the `kind` cluster if needed, bootstraps the operators, builds and loads the images, installs the chart, seeds the demo data, and prints the **front door** and **demo logins**.

```
./install.sh --defaults    # non-interactive, all bundled (CI / quick)
./install.sh --uninstall   # remove the release (keeps the cluster)
```

The wizard asks, in order: the **target cluster** (`kind` / `stackit` / `other`); per-backend **bundled or external** for Postgres, OpenSearch, object storage and cache; and the **LLM endpoint** (local mock, or a real provider whose key is stored as a Kubernetes secret reference, never inline). Anything other than *bundled* is written into a small generated overlay as `mode: external`.

3.3 Open the front door

Everything is reachable by port-forward. Start with the OS UI:

```
# OS UI – the product front door (Home, Cockpit, Strategy, ... , Governance)
kubectl -n agentic-os port-forward svc/os-ui 8080:3000    # http://localhost:8080
```

The **OS UI** is a Next.js app with a left sidebar. Every surface calls the in-cluster backends through **server-side API routes**, so credentials and keys never reach the browser. The stack's operational console is **embedded inside the OS UI** at **Platform → Components** – it reads the in-cluster Kubernetes API and the baked-in component docs natively, so there is no separate admin-console service to port-forward. Locally there is no login; on a real deployment you sign in with your Ory identity.

On a real deployment the back-end tools no longer need their own port-forward or login: they load **same-origin, inside the OS UI**, proxied by the Node server at `/tools/<tool>`. The server turns your OS session into whatever the upstream tool needs – **Level-1 header SSO** injects your identity (`X-Forwarded-User` , plus a tool-specific alias like Forgejo's `X-WEBAUTH-USER`) mapped through your per-

domain role, and the tool **auto-provisions** the matching account on first request. Credentials never reach the browser, and the proxy pins `frame-ancestors 'self'` so only the OS shell can embed the tool. (The port-forward table below is the local-dev path.)

The handful of back-end consoles you may want first (the full table is in the Reference):

CONSOLE	PORT-FORWARD (<code>KUBECTL -N AGENTIC-OS ...</code>)	URL	LOGIN
OS UI	<code>port-forward svc/os-ui 8080:3000</code>	<code>http://localhost:8080</code>	– (local)
Langfuse (traces)	<code>port-forward svc/agentik-os-langfuse-web 3000:3000</code>	<code>http://localhost:3000</code>	<code>admin@datamasterclass.com</code> / <code>langfuse-local-dev-admin</code>
Superset (BI)	<code>port-forward svc/agentik-os-superset 8088:8088</code>	<code>http://localhost:8088</code>	<code>admin</code> / <code>superset-admin-local-dev</code>
Forgejo (git)	<code>port-forward svc/forgejo-http 3001:3000</code>	<code>http://localhost:3001</code>	<code>gitea_admin</code> / <code>forgejo-admin-local-dev</code>

These are local dev throwaways (profile `local`), clearly marked as such and never reused on STACKIT, where every secret is external. See the *Security model* chapter.

3.4 Bootstrap: the first administrator, then real users and roles

On a real deployment, the platform starts **closed**. Getting from an empty install to a working team is a deliberate, audited sequence – this is the secure first-run path:

- 1. Claim the first administrator.** Identity is backed by **Ory**; the secure first-run bootstrap creates exactly one tenant **Administrator**. This account **auto-verifies** – no email server is required to start using the platform – and no password is ever shown in the UI; the Admin sets credentials through the identity flow.
- 2. (Optional) Wire up email** for later verifications and invites. The platform sends through one of two transports (precedence **Graph** → **SMTP** → **none**), sender `support@datamasterclass.com` by default:
 - **Microsoft Graph** `sendMail` (recommended for Microsoft 365) – an **Entra app registration** with the Graph **Application** permission `Mail.Send` (+ admin consent), called via OAuth2 client-credentials. This avoids SMTP basic-auth, which M365 is deprecating.
 - **SMTP fallback** – a minimal built-in client; set `SMTP_HOST` (+ port / user / pass / secure). Use this for any generic mail relay. With neither configured the platform simply runs without email; the bootstrap admin already auto-verified, and later accounts can be verified out of band.
- 3. Open Admin.** As that Administrator, go to **Platform** → **Admin** (`/platform` , hard-gated to admins). The **Overview** cockpit shows component health, spend versus the tenant envelope, users and

domains.

4. **Create your domains.** In **Admin → Domains**, create a domain per team or business area, and toggle its optional layers (for example, turn Science on for a domain that needs ML).
5. **Invite real users.** In **Admin → Users & Access** (or **Governance → Users & access**), invite people via Ory – you send an invitation, never a password – and give each a **role per domain**.
6. **Roles take effect everywhere.** Each assignment **compiles to OPA**, so a person who is a *Builder* in one domain and a *User* in another sees exactly the right controls in every tab, immediately.
7. **Set the guardrails.** Still in Admin, set the **model allowlist and defaults**, the **egress allowlist**, and the **cost envelope**. These compile through the same policy engine the tabs enforce – configuring a right here is never a governance bypass.

From here, each person opens the **OS UI**, lands on **Home**, and follows the golden path for the work they need to do – the rest of this guide.

3.5 Try the seeded demos

Four end-to-end demos ship seeded, so the system proves itself the moment it is up:

```
# 1. Ask the RAG agent – retrieve (OpenSearch) → generate (LiteLLM) → trace (Langfuse):
kubectl -n agentic-os run ask --rm -i --restart=Never --image=curlimages/curl:8.11.1 -- \
  curl -sS http://sample-agent:8000/ask -G \
  --data-urlencode "q=What provides the retrieval backbone?"
```

The reply includes the answer, the retrieved knowledge titles, and `traced_in_langfuse: true` – open Langfuse to see the run. The other three: **query the lakehouse** (the governed `query` tool over central Trino), **build a dashboard** in Superset on the seeded `daily_revenue` dataset, and **ship software** (push to the seeded `demo-app` repo → Forgejo CI builds an image → Argo CD redeploys). Each has a one-card launcher on **Home**.

4 Using the platform, tab by tab

Every chapter below follows the same shape: **What it's for** • **The golden path** • **Roles** • **Connects to**. They are in **sidebar order** — the business tabs first, then a short **Platform** section for the operational consoles. Most flows run live against the cluster and fall back to a labelled offline mock on **kind** — a ✓ always means a real apply and verify.

4.1 Home — the golden-path launcher

What it's for. The warm front door after you pick a domain. Home **only orients and routes**: an illustrated launcher of the ten golden paths, with copy and dimming that shift by persona. The live "what's moving / what needs me" view now lives one click away in the **Cockpit** — Home no longer carries the cockpit modules.

The golden path.

1. Land on the editorial hero with your name, **domain**, and persona badge.
2. Scan the **Golden paths** launcher — ten illustrated cards (Data, Knowledge, Agents, Software, Science, Metrics, Dashboards, Big Bets, Marketplace, Connections), each with a role-aware primary action. Cards your role can't act on yet are explained but dimmed.
3. Click a card's action to deep-link straight into that tab's flow — or its tutorial link to learn it in place.
4. Use the **"Your cockpit"** call-to-action to open the live cockpit when you want to see what's moving and what needs you.

Roles. All four — the launcher copy and dimming reorder by persona. **Connects to.** Routes into all ten path tabs; hands off to **Cockpit** for the live view.

4.2 Cockpit — what's moving, what needs you

What it's for. The live, governed overview that used to sit under Home. A persona-ordered cockpit of your work: a headline pulse strip, the working modules, and a scannable **top-items-per-artifact** board. Cockpit *reads and routes* — it never recomputes another tab's numbers and never bypasses governance.

The golden path.

1. Read the hero greeting and the **pulse strip**: *Needs you* • *In progress* • *Your items* • *Spend vs. cap*, each linking into its owning tab.
2. Work the persona-ordered modules under **"What's moving"** — *What needs me* • *My WIP* • *Domain pulse* • *Health & cost* • *Recent activity* • *Ask anything* — reordered to your role.
3. Clear an item from **What needs me** (an approval or draft) or pick up **My WIP**.
4. Scroll to **"Top items, by type"** — the most-notable thing you can see in each part of the registry, governed and scoped (never another domain's, never someone else's drafts).
5. Use **Ask anything** to get an answer or scaffold a Personal draft (promotion stays human and traced).

Roles. All four — the modules and ordering shift by persona; everything is OPA/RLS-scoped. **Connects to.** Reads Governance, the artifact registry, Strategy and Monitoring; routes into every tab. (*Domain pulse and Health & cost read live feeds, mock-stubbed on **kind** and labelled as such.*)

4.3 Strategy — pillars, value and adoption

What it's for. Where the company plans its agentic transformation, in exactly **three sections**, top to bottom. Calm and Apple-grade; governance stays server-side.

The three sections.

1. **Big Bets** — your strategic **pillars**, shown **side by side**, each holding the big bets that deliver its business value. Each pillar shows its current value + metric, its bets (with ready / in-progress / planned counts) and inline create/edit. A bet box opens the bet detail (below).
2. **Self Service** — how broadly your people build for themselves: **Total Users • Analytics • AI • Software • Builders • Creators** — distinct creators by capability area plus the builder/creator population, scoped to the viewer's company/domain.
3. **Foundations** — the governed asset base: **promoted + certified** artifact counts **per type**, the certified backbone every bet builds on.

The old pillars/targets/value-rollup boxes and the RLS explainer are **gone**. A pillar's **value metric** is described here in business terms, then kept one of two ways: **set up as a governed Cube Metric** (hand off to Metrics) or **tracked manually each month** (a small value entry feeds the pillar's history chart).

The bet detail (opened from a pillar's bet box): a top row of **Value** + Planned / In-progress / Ready counts, then **Value metric over time** → **Roadmap** (each component on its due-date timeline, with a go-live marker) → **Components** (each one a box that deep-links to *Edit in*) → **Audit** (a subtle governance footer). There is **no Composition** view.

Roles. Administrators define **company** (tenant) pillars; Builders define **domain** pillars; Creators and Users view. **Connects to.** **Big Bets** (each bet links up to a pillar), **Metrics** (the governed Cube metric a pillar can track), every Build tab (the foundation counts), **Monitoring**.

4.4 Big Bets — initiative roadmaps

What it's for. A strategic AI bet as a **goal + dated roadmap** built from real artifacts across the platform, linked up to a Strategy pillar with a value target.

The golden path.

1. Browse the **Portfolio, grouped by strategic pillar** — each pillar is its own section of bet cards (realized vs. target value, a completion bar, component count, go-live date, at-risk flag).
2. **New Big Bet** opens a drawer whose fields are, in order: **Owner** → **Strategic Pillar** → **Problem Statement** (required) → **Solution Idea** → **Value** (the chosen pillar's metric + currency, with a target) → **Planned Go-Live**. Choosing the pillar carries its metric onto the value field.
3. Open the bet → its detail is **Value** → **Roadmap** → **Components** → **Audit**, with **Archive** — and **no Composition**. Add **components** by linking existing artifacts or scaffolding new ones through each

tab's governed flow (the planner proposes a breakdown but never ships).

4. Watch **status derive live** from each artifact's real lifecycle (planned / in-progress / ready / blocked); the roadmap rolls up on-track / at-risk and flags an unrealistic go-live.

Roles. Anyone can draft a bet; advancing and promotion stay human (Builder/Admin); the planner runs as a non-promoting actor. **Connects to.** **Strategy** (pillar up-link), every Build tab (components), **Monitoring** (runtime health). *(Cross-tab component sources are offline-mock on `kind`, honestly labelled.)*

4.5 Agents — compose, govern, run

What it's for. One page where a domain's **agent systems** (instructions + tools + memory) are composed, governed and run, plus a strip of the **deployed agent systems** currently running. There are no sub-tabs. Every model and tool call goes through the gateway — no agent ever touches a raw resource.

The golden path.

1. Author an agent system three equivalent ways — a drag-and-connect **canvas**, **Monaco** editing of `system.yaml`, or the **agent-system assistant** chat — all editing the same Forgejo-versioned file.
2. Grant the **resources** (data products, knowledge, files, connections) and **tools** the system may use; a validation gate must pass first.
3. Press **Build** — *Build = execute + verify*: it runs the compiled system and checks it, every call routed through **LiteLLM** → **OPA** → **Langfuse**. Pick a LiteLLM model per agent.
4. **Run / schedule / toggle** the system, or **fork-to-own** a copy. By default agents are *propose-don't-commit* — a write pauses for approval and enqueues in **Governance**.
5. Below, the **Deployed agent systems** strip shows what's running (the seeded Sales/Finance assistants; the **poem agent** is a fun demo, not infrastructure).
6. Certify a finished agent to list it in the Marketplace.

Roles. Creators and Builders author; promotion and held write-backs are human-gated. **Connects to.** **Knowledge** (attach-as-context, scaffold-from-workflow — both live), **Connections** (a connection becomes a tool), **Governance**, **Monitoring**, **Marketplace**. *(Build/Run execute against the live agent-runtime when a cluster is reachable, else an in-process mock — labelled either way.)*

4.6 Software — build governed apps, sovereign

What it's for. One page: a big home-style "Create new software app" launcher, then your **running apps** as clean tiles. Git is **in-cluster Forgejo (sovereign)** — a repo is **auto-created in-cluster** on create; there are **no GitHub, no accounts, no tokens**, and your code never leaves.

The golden path.

1. Click **Create new software app**: name it and pick a template (web app / service / script / dashboard). It auto-provisions a sovereign **Forgejo repo in-cluster** and drops you straight into the app's **build chat beside a self-hosted Monaco editor** (`/software/{id}?mode=edit`).

2. Iterate in the **Claude-style build chat**: describe what you want; the agent writes the code and commits to the app's own repo, and you can edit files directly in the code editor.
3. Open any running app's tile → its **app page** offers two modes:
 - **Monitor** — live status, "**Open app UI**" / "**Show API details**", and **Publish**.
 - **Edit** — the build chat + code editor again.
4. **Request deploy** assembles a **review card**: the security scan (SAST • dependencies • secret-scan), the requested **envelope** of governed resources, the cost/resource footprint, and the change diff. It enqueues to Governance under **Deploy reviews**.
5. A **Builder/Admin in the app's domain** decides it; a leaked secret or a high/critical finding **blocks go-live**, and a creator cannot self-approve. Routine **in-envelope** updates auto-deploy; anything that broadens scope re-opens the review.

Committing the app **auto-creates its MCP** from its OpenAPI spec (**reads on, writes held for approval**), governed by the same OPA gate every connection uses — so the app is instantly usable as a tool by your agents.

Roles. Any owner previews; a Builder/Admin-in-domain approves go-live; an Administrator certifies to the Marketplace. **Connects to. Connections** (the auto-MCP), **Governance** (deploy reviews), **Data** ("Use as Data"), **Marketplace, Monitoring**.

4.7 Science — classic ML (opt-in, Layer 4)

What it's for. Taking **traditional ML** (regression, classification, forecasting, clustering — *not* LLMs) from a governed data product to a governed, deployed **model-as-service**. Off by default; GPU is optional and cost-gated.

The golden path.

1. If the tab is off, a disabled surface explains that an **Administrator** enables ML per domain (`ML_ENABLED=true`).
2. When on, check the **Layer-4 stack** health grid (Featureform, MLflow, Dagster, KServe, JupyterHub).
3. Walk the guided path on the seeded slice: **Explore** → **Features** → **Train** → **Register** → **Certify** → **Deploy** → **Consume** → **Monitor** (an ML agent drives the no-code common path; JupyterHub is the escape hatch).
4. Move up the **tier ladder** (Personal → Shared → Marketplace): a **Builder** promotes and go-lives Staging → Production; an **Administrator** certifies, choosing read-in-place or fork-allowed.
5. A deployed model exposes **two front doors from one KServe endpoint** — a governed REST `predict` API and a governed `predict` MCP tool — both through OPA + LiteLLM, capped and traced.
6. Watch **drift** (PSI vs. AUC) and trigger a **retrain**. The ML agent proposes only — it is hard-blocked from certify and go-live.

Roles. Administrators enable + certify; Builders promote/go-live; Creators build/train; the agent proposes. **Connects to. Data** (the governed mart), **Software/external** (REST), **Agents** (MCP), **Monitoring** (the same drift signals), **Marketplace**.

4.8 Knowledge — the domain's operating manual

What it's for. The domain's human-authored manual: general domain knowledge plus a **workflow** per business process, made retrievable by a knowledge agent behind document-level security.

The golden path.

1. In **Domain knowledge**, edit the four guided sections (overview / glossary / goals / context) — auto-pinned as the base context for every domain agent. The **knowledge agent** can draft; you paste the result in.
2. Switch to **Workflows** and browse tiles (Personal / Shared / Marketplace).
3. **+ New workflow** → name a business process → open the detail editor: an **actor-coloured swimlane** whose lanes (Human / Software / Agent) are **vertical columns and flow top → bottom, Monaco markdown**, and a derived diagram — all editing one versioned `workflow.md`.
4. Capture per-step actors, inputs/outputs and **links** (a gap jumps you to where to build it); mark a decision rule **hard** to compile it into an **OPA guardrail**.
5. A **Builder/Admin publishes** a draft to make it live; **certify** lists it as a knowledge product.
6. Hand a certified workflow off to **scaffold a domain agent**.

Roles. Creators author Personal drafts; Builders/Admins publish and certify; document-level security keeps Personal units private. **Connects to.** **Files** ("Use as → Knowledge"), **Agents** (context pack + agent scaffold), **Marketplace**, **Governance**.

4.9 Files — a calm governed drive

What it's for. A drive for any **unstructured** file — documents, images, video, audio, archives — auto-indexed so agents can search and cite it, governed exactly like Data.

The golden path.

1. In **Files**, **upload** anything (drag-drop, any type) and watch the status chip move *Processing* → **Searchable** ✓. Behind it: parse (Docling for docs, transcription for A/V, OCR/caption for images) → embed → hybrid OpenSearch index.
2. Organize with **folders + tags** and preview in place.
3. **Search** across names, tags and content (DLS-filtered to what you may see).
4. In **Sources**, connect **Google Drive / OneDrive**; synced files **re-govern** under our tiers, not the source's ACLs.
5. **Promote/certify** up the ladder (a light docs gate: owner + description + at least one tag); `restricted` files are stored but never indexed.
6. **"Use as"** distils a file into **Knowledge** (a tacit note) or **Data** (a guided Bronze import), with lineage recorded.

Roles. Reuses Data's gates — Creators upload Personal; Builders promote; Administrators certify.

Connects to. **Knowledge** and **Data** (Use-as), **Connections** (Drive/OneDrive), agents (the `files_retrieve` tool), **Marketplace**, **Governance**.

4.IO Data — datasets, refined and governed

What it's for. Turning a plain-language flow — refine a dataset **Bronze** → **Silver** → **Gold**, then share it — into real governed artifacts (a dlt pipeline, dbt models, a Cube cube), with no YAML for non-technical users. The tiles are grouped **Data** (your Personal datasets) • **Shared Data** (domain) • **Marketplace Data** (certified, cross-domain).

The golden path.

1. Open the **Datasets** tab and create or open a dataset; it presents as guided **Bronze / Silver / Gold** panels — one logical dataset in three versions.
 - **Bronze** — bring it in: upload a file or pull a masked slice of a product.
 - **Silver** — clean it up: fix types, drop duplicates, set the key.
 - **Gold** — make it ready: harmonize to the shared shape and add quality checks.
2. Press **Build** on a stage — it runs that stage's adapters (apply + **verify**); the row turns ✓ only when both pass. Toggle "**Show the code**" to see the real artifact.
3. Work privately in your **Data** tier (a fast DuckDB sandbox); promoted data lives in Trino/Iceberg.
4. Browse structured assets in **Catalog** (OpenMetadata) and **Preview** any into Query.
5. Ask questions in **Talk to your data** — a RAG agent over your domain's data (LangGraph + OpenSearch + LiteLLM, Langfuse-traced) — or run SQL in **Query**.
6. Share like a review: **request promotion** (Creator) → a **Builder** promotes the dataset to **Shared Data** → an **Administrator** certifies it to **Marketplace Data** (a product).

Roles. Creators create datasets; Builders promote to Shared Data; Administrators certify to Marketplace Data (approvals run through Governance). **Connects to.** **Metrics** (the Gold auto-cube), **Dashboards**, **Knowledge/Files** ("Use as"), **Marketplace**, **Governance**.

4.II Metrics — one number, everywhere

What it's for. The KPI semantic layer. Define a measure once and "Revenue" or "Active customers" resolves to the **same number** in the explorer, in dashboards, and in the agent's `metrics` tool.

The golden path.

1. Open the **Registry** and browse every governed metric (tier-honest).
2. **Define** a measure three convergent ways — a friendly **form**, the **metrics agent** ("define revenue on sales"), or hand-edited **Cube YAML** — all producing the *same* canonical member (e.g. `Sales.revenue`).
3. **Explore** it: slice by dimensions with no SQL, under your own row-level security (two viewers see different rows); drop to SQL/Trino if you need to.
4. **Govern** it: promote Personal → Shared (Builder) → Marketplace (Admin); promotion runs a consistency check (documented, defined, resolves on its member).
5. Inspect quietly in **Live Cube**. The metric now feeds Dashboards and agents identically — and can be the governed value metric behind a Strategy pillar.

Roles. Creators define; Builders promote; Administrators certify. **Connects to.** **Data** (the base cube), **Dashboards** and **Agents** (consume the member), **Strategy** (pillar metrics), **Marketplace**.

4.12 Dashboards — governed BI

What it's for. Apache Superset dashboards built read-only on governed Cube metrics, so the BI layer and the agents can never disagree.

The golden path.

1. Open **Dashboards** — tiles grouped Personal / Shared / Marketplace.
2. Build one two convergent ways: **drag charts** in Superset, or ask the **dashboard agent** ("build me a Sales overview") — both edit the same dashboard, on the same metrics.
3. **Double-click** a tile to open it inline via the Superset Embedded SDK; a server-minted **guest token carries the viewer's RLS**, so a shared dashboard still shows only your rows.
4. Set a **threshold alert** on a metric → notify by email/Slack/in-app *and* optionally trigger a governed agent run (Langfuse-traced).
5. Schedule a **report** (a dashboard snapshot on a cadence).
6. **Promote/certify** the dashboard up the ladder without ever broadening rows.

Roles. Creators build; Builders promote to Shared; Administrators certify to Marketplace. **Connects to.** **Metrics** (consumes members, never defines), **Agents** (alert-triggered runs), **Monitoring** (traces), **Marketplace**.

4.13 Connections — governed bridges to outside systems

What it's for. External systems only. A **Connection** is a governed bridge to an *outside* system — `credentials + endpoint + a set of governed tools`, never a raw pipe — used two ways: to **bring data in** (Database / API / SaaS → dlt → Bronze) and to **expose external APIs/MCPs as tools** for agents and software. You grant **use**, never the token: the secret stays in the secrets store. (Platform-service status moved out — that now lives in **Components**.)

The golden path.

1. In **My connections**, register a connection (source type + endpoint) under the Personal → Shared → Certified lifecycle. Any user can connect their **own** account (e.g. their Google Drive) via per-user OAuth; Builders/Admins create **shared** connections with service credentials.
2. In **Registry**, see the supported connectors (available vs. on the roadmap) and the auto-generated **App MCP connections** (one per Software app).
3. In **Governed connections**, set each tool's **capability profile** — a mode of *Off / Read / Write-approval / Write-bounded / Blocked*, plus scope, rate and cost limits. **Reads on, writes off by default**; the profile compiles to an OPA policy, and a grant to an agent can only *restrict*, never broaden.
4. In **Build a connector**, describe a source and the connections agent drafts the connector and the credentials it needs.
5. A write pauses for approval: **with a human present**, inline with a before/after preview; for **autonomous agents**, bounded by a Builder-set safety preset, with anything out of policy blocked and queued in Governance.
6. New egress endpoints are **Builder-request → Admin-approve**; all outbound traffic is logged.

Roles. Any user adds personal connections; Builders/Admins add shared credentials and attach connections to agents; Administrators manage the egress allowlist and Marketplace. **Connects to.** **Agents** (connection = tool), **Data/Files** (connection = source), **Software** (the auto-MCP), **Governance** (egress + write-back approvals), **Marketplace**.

v1 status – read this. The **Governed connections** tab is the real capability-profile and policy surface and works as described. Some **My connections** / **Build a connector** create flows are **scaffolded in v1**: they draft a configuration for review rather than minting live credentials. The chapter describes the full design so the shape is clear; expect those flows to say "scaffolded in v1" on screen until they are wired.

4.14 Marketplace — consume across domains

What it's for. The *Consume* counterpart to every Build tab's **certify** step: discover and reuse **Administrator-certified products of every type** across the tenant's domains. Importing is a **governed grant**, not a copy.

The golden path.

1. Browse the cross-domain catalog; filter by **type**, domain or tag, or search.
2. Read each listing's **source pill** (live / offline-mock) and its certification badge, owner, lineage, quality and usage signals.
3. Open a listing → an RLS-filtered **preview/sample**, lineage and ratings.
4. **Import.** The default is **read-in-place**: you consume under **your own identity + row-level security**, and the owner's certified artifact stays the source of truth. Some types differ – an **app** deploys your own instance, a **connection** is a template (bring your own credentials), an **agent** is fork-to-own.
5. Approval-required imports create a pending grant + a **request in Governance**; clearing it activates the grant.
6. Track grants in **My imports**, rate 1–5; Administrators **deprecate** lineage-aware (importers are warned; in-use grants are kept).

Roles. Administrators certify (upstream) and deprecate; anyone discovers and imports. **Connects to.** Every Build tab's certify step, **Governance** (import approvals), **Monitoring**.

4.15 Monitoring — artifact observability

What it's for. The read/observe plane for your **artifacts**: trace runs, watch spend, and surface pipeline + model **drift** – scoped to your identity. It is strictly **read-only**; it does not set policy or caps (that's Governance), it does not watch business KPIs (that's Dashboards), and **infrastructure health lives in Platform → Components** – it is deliberately *not* a Monitoring lens.

The golden path.

1. Read the **scope pill** (your Ory→OPA identity).
2. Scan **Needs attention** – the few red/amber items that need a human lead, not a wall of green.
3. Review the **four lenses**: **agent & run** observability (Langfuse), **data-pipeline** health (Dagster + dbt), **cost & usage** vs. the Governance caps (LiteLLM), and **artifacts** across all tabs including ML

(freshness, lineage, **drift**).

4. Click any run/pipeline/model → the **trace drawer**: the full Langfuse trace (steps, tool calls, the context pack, inputs/outputs) plus logs – gated to your scope.
5. Follow the **correlation spine** – run ↔ pipeline ↔ artifact – and cross-link to the Governance audit entry and the cost cap it spends against.

Roles. A User sees only their own runs/cost; a Builder their domain; an Administrator the tenant. A User cannot open another user's trace (enforced server-side). **Connects to.** Reads from every tab; reads Governance caps; defers KPI alerts to Dashboards, cap-setting to Governance, and infra health to Components.

4.16 Governance — the control plane

What it's for. Consolidate, decide, record. Governance *enforces* policy and *executes the effect behind a decision* – but it doesn't author tenant structure (that's Admin).

The golden path.

1. Work the **Approvals inbox** – one scoped queue of every side-effectful, role-gated action: a Software deploy review, an autonomous out-of-policy action, an access/import request, a new egress endpoint, a promote/certify. **Approving is the action:** on approve the platform runs the effect (Argo deploy • policy grant • egress allowlist • promote • the queued run) and writes the audit.
2. Read the consolidated **Policies** plane; an Administrator can override (revoke a grant).
3. Search the hash-chained **Audit** (who / when / why) and verify chain integrity.
4. Set a **cap** in **Cost & limits** – over-cap is enforced.
5. Manage **Users & access** – invite via Ory (raw passwords are rejected) and assign role-per-domain, compiled to OPA.
6. Use **"Approve & remember"** to turn a decision into an editable standing policy.

Roles. Users see and act on their own requests; Builders their domain's queues, policy, audit and memberships; Administrators the whole tenant. **Connects to.** Software, Connections, Data/Files, Agents/Science and Marketplace all raise cards here; **Admin** compiles the identity Governance reflects; **Monitoring** watches.

4.17 The Platform section

The sidebar closes with a small **Platform** group of operational consoles – most gated to Builders or Administrators.

4.17.1 ADMIN — THE TENANT CONTROL ROOM

What it's for. A tenant-scoped, **Administrator-only** area above the per-domain workspace, where the tenant's structure, identity, models, egress and posture are authored – all of which compile through to OPA. Labelled **Admin** in the sidebar (the conceptual "Platform Admin").

The golden path.

1. Open `/platform` (hard-gated to admins) → the **Overview** cockpit: component health, spend vs. envelope, users, domains, and the principals compiled to OPA.
2. Work the **open admin alerts**, then jump via the quick links.
3. **Domains** — create, rename, archive or transfer; toggle a domain's optional layers (this scales already-provisioned workloads 0↔1; the UI never provisions cloud resources).
4. **Users & Access** — invite via Ory (the credential is never returned) and set roles.
5. **Models & Providers / Security & Egress / Cost & Billing / Backups & Restore** — configure; provider keys are stored as a **reference + fingerprint**, never raw.
6. Destructive actions (restore, disable) require a **typed-confirmation guard** and are audited; identity/domain/egress/model changes **re-compile to OPA**.

Roles. Administrators only. **Connects to.** **Governance** (enforces and shows the compiled plane), **Components + Monitoring** (watch live health and spend), and every tab (via the OPA grants the compiler emits).

4.17.2 COMPONENTS — PLATFORM SERVICES & SYSTEM HEALTH

What it's for. The operational console for the **stack itself**, embedded in the OS UI — the home of everything that moved out of Monitoring and Connections. It lists **every platform service** with **live system/cluster health**, lets you switch toggleable workloads on/off (scale 0↔1), and surfaces each one's address, login, docs, and its place in the **dependency/impact chain** (what a toggle affects).

The golden path.

1. Open **Platform → Components**; services are grouped **by layer** (L1 core, L2 foundations, infra, L3 self-service, L4 science, security & platform), each with a live status dot.
2. Read a service's **summary, address (port-forward + URL), login and docs**; core services are marked always-on and cannot be switched off.
3. **Toggle** an optional workload on/off (scale 0↔1) — the dependency/impact view shows what else is affected. The routes read the in-cluster Kubernetes API and the baked-in component docs natively; the browser never touches a Kubernetes credential.

Roles. Builders/Administrators. **Connects to.** **Admin** (structure + layer toggles), **Monitoring** (artifact observability sits beside this infra view).

The remaining Platform entries — **Users, Gateway** (the LiteLLM model/MCP gateway), **Orchestration** (Dagster), **Workbench** and **Terminal** (Builder/Admin developer surfaces), **Consoles** (a launchpad for the full external tool UIs, now embedded **same-origin** through the OS UI at `/tools/<tool>` with Level-1 SSO), and **About / Licenses** — are thin windows onto their in-cluster services and the license set.

4.18 Tutorials — learn any path in place

What it's for. One illustrated, hands-on tutorial per golden path, authored once and reached two ways — Home's launcher and a tab header's **"Tutorial"** — that resolve the *same* registry entry, so they never drift.

The golden path.

1. Trigger a tutorial (a Home launcher card, or a tab header's "Tutorial").
2. Read the **Hook**, then 3–5 illustrated steps in the overlay (your route and scroll are restored on close).
3. Choose **"Walk me through it"** → live **coach-marks** spotlight the real controls, located by stable `data-tutorial-anchor` ids so the highlight can't desync.
4. **Practice** in sandbox mode on the tab's personal lane – every governed write is stripped, so nothing persists.
5. **Graduate** to do it for real, where OPA and RLS govern every step.
6. Finish on **"You did it"**, with cross-links to the next paths.

Roles. Framing is role-aware (Creator "create", Builder also "review/promote", User "use"); the core path is identical and OPA/RLS are never bypassed. **Connects to. Home** (the launcher gallery) and each path tab via its anchors. *(Anchors are wired incrementally – Data and Agents today; a tab without anchors degrades gracefully to an "open this tab" card.)*

5 Reference

5.1 Deploying to your cloud (STACKIT)

Locally everything is self-contained. On **STACKIT** (or any cloud) the platform runs the **full Layer 1–4 stack** – the **Data engine** (central Trino over the Iceberg lakehouse), the **metrics layer** (Cube), **Science / ML** (MLflow + KServe + Featureform + JupyterHub), **Orchestration** (Dagster), the developer **Workbench** and **Terminal**, and sovereign git via in-cluster **Forgejo + Argo CD** – alongside L1 (LangGraph/LiteLLM/OpenSearch/Langfuse) and L2 (OPA/Docling/Haystack/dbt/ OpenMetadata). It is the **same chart**; switching a backend to a managed service is only a values choice (`values.stackit-managed.yaml`), and the heavier layers (Science, Terminal, Workbench) ship **pinned and provisioned but off by default**, enabled per domain when needed.

1. **Prerequisites.** A STACKIT organization + project in **EU01 / Deutschland Süd**, and a service-account key with provisioning roles (SKE + Object Storage + DNS), saved as `stackit/sa-key.json` (gitignored). **This key is the gate for any live deploy** – you can build and validate the entire chart on local `kind` with no key.
2. **Provision the managed resources** (Terraform preferred): an **SKE cluster** (CNI = Cilium), a **node pool** sized for the full stack ($\approx 3\times$ g1.4 for L1+L2, 4–5 $\times$ for L3, more if Science is on), **Object Storage** buckets + S3 credentials, a **load balancer + public IP**, a **DNS zone**, and **Secrets Manager / KMS**.
3. **Bootstrap the in-cluster platform** before the OS chart: ingress-nginx + cert-manager, the SKE storage class, Cilium default-deny egress, the External Secrets Operator, CloudNativePG, Velero, and Argo CD.
4. **Point the OS at managed backends** in `values.stackit-managed.yaml`: object storage and Postgres to STACKIT, the LLM to **STACKIT AI Model Serving** (`llm.mode: external`, `provider: stackit`), Trino → the Polaris REST catalog (OAuth2), plus ingress hostnames (Forgejo, Superset, the OS UI), the egress allowlist and per-domain quotas. You can mix freely – managed Postgres but bundled OpenSearch, for example – and turn Science / Terminal / Workbench on per domain.
5. **Deploy and verify:**

```
helm install agentic-os charts/sovereign-agentic-os -n agentic-os --create-namespace \
-f values.stackit-managed.yaml -f values.generated.yaml
```

Point DNS at the load balancer, confirm the consoles, confirm the default-deny egress baseline is active, then create your first domains.

Cost. Roughly **€450–670/mo** for L1+L2 at typical sizing; the full L3 + Science stack scales it up. **Scale the node pool to zero between sessions** (storage + IP persist at $\sim$ €16–20/mo). LLM token spend is separate and **capped in LiteLLM**.

5.2 Sizing & capacity

Three different resources get confused under one word, "size." They scale for different reasons, so it helps to keep them separate. The verified single-node deploy runs on a STACKIT `m3i.16` worker — **16 vCPU / 128 GB RAM** — and in practice RAM ran at only **~2–4%**: memory is plentiful. The resource that actually bit us was the **node disk**, and it bites for a reason that has nothing to do with how much data you have.

RESOURCE	THIS DEPLOY	HOLDS	HOW IT SCALES
Node RAM	128 GB (<code>m3i.16</code> , 16 vCPU)	Running pods — Trino's JVM heap, OpenSearch, the in-box model	With concurrency / workload , not data. Ran ~2–4% here.
Node disk	200 GB (was 80)	Container images + local model weights — all Layer 1–4 images (~40–60 GB) + the in-box model (Ministral ~3 GB; a Magistral 24B would add ~15 GB) + image-churn headroom	FIXED . It does not grow with your dataset. Size it once for images + models.
Data storage	Object storage + PVCs	The Iceberg lakehouse on object storage (in-cluster MinIO for the demo → STACKIT Object Storage / S3 for real scale → TBs), plus PVCs for OpenSearch (indices + embeddings), Postgres (metadata), ClickHouse (traces), MLflow (artifacts)	Independently , with the dataset. "More data" lands here.

The one gotcha worth internalizing: **don't confuse node RAM (128 GB) with the node disk** (the small volume that fills). The original 80 GB disk filled with images during deploy → **disk-pressure** → the node was **cordoned** → pods could no longer schedule. The fix is a **200 GB** node disk (`node_volume_size_gb`, now the Terraform default), sized for images + model weights with churn headroom — **not** for data.

And the corollary that keeps cost honest: **real data never touches the node disk**. It lives on separate, independently-scalable storage. When the dataset grows into the terabytes you grow the **object storage and the PVCs** — the node volume stays exactly where it is. More data ≠ a bigger node disk.

5.3 Durable lakehouse

On a long-lived deployment the lakehouse must survive pod restarts and node-rolls. Two backends hold the data and both are now persistent: **MinIO** keeps the Iceberg data files on a **PVC**, and **Polaris** keeps its catalog metadata in a **relational-JDBC metastore** (the bundled, PVC-backed Postgres) instead of in-memory — so the `lakehouse` warehouse registration is not lost on restart (otherwise Trino and the `query` tool report *"Unable to find warehouse lakehouse"* even with MinIO persisted). A `polaris-catalog-init` **Job** registers the `lakehouse` catalog on deploy, so queries resolve it with no manual step. (On a throwaway `kind` box the default stays ephemeral by design.)

5.4 Security model

The platform ships **secure by default**. Four guarantees hold throughout:

- **Default-deny egress**. NetworkPolicies deny outbound traffic except DNS, intra-namespace traffic, and the API server; only the **egress proxy** may reach the internet, and it is allowlist-only and logs everything. (On `kind` the kindnet CNI doesn't enforce NetworkPolicies, so the app-layer chain —

OPA → proxy → `web_fetch` — provides the guarantee; on STACKIT, **Cilium** enforces them and adds FQDN-aware allowlists and DLP.)

- **OPA tool authorization, least privilege.** A principal may invoke a tool only if granted; unknown principals and ungranted tools are denied. Internet access is an explicit grant — `web_fetch` is ungranted by default. Agents use a **scoped** virtual key with a spend cap, never the master key.
- **The web is data, not instructions.** The only path out is the governed `web_fetch` tool: OPA-authorized, routed through the egress proxy, and returned **as sanitized data** — never auto-written into the knowledge base. Retrieval results and tool output are treated the same way (the prompt-injection posture).
- **No real secret in git.** `.gitignore` blocks secret patterns; the chart ships only secure defaults. The local dev passwords in this guide exist only under `profile: local`. On STACKIT every secret lives in **Secrets Manager / KMS** and is synced by the **External Secrets Operator**; the chart references secrets by name only. Outbound-email secrets (the Microsoft Graph token / the SMTP password) are held the same way and are never logged or returned.

Every agent action — model calls, tool calls, retrievals — is **traced in Langfuse** with token cost and latency; outbound requests are logged at the egress proxy; telemetry/phone-home is disabled for a sovereign, offline posture.

5.5 Models & the LLM gateway

Every model call goes through **LiteLLM** — the one gateway that enforces the model allowlist, per-key spend caps, tracing and graceful back-pressure. The self-hosted default routes are **free**; a live STACKIT deployment fronts two pay-per-token open-weight **Qwen** tiers on **STACKIT AI Model Serving**, matching the assistants' two phases:

- **Reasoning / planning** — `Qwen/Qwen3-VL-235B-A22B-Instruct-FP8` (LiteLLM `sovereign-reasoning`), the PLAN phase of every assistant.
- **Execution / default** — `Qwen/Qwen3.6-27B` (`sovereign-default`), tool-calling and general chat.

Qwen "thinking" is disabled (`chat_template_kwargs.enable_thinking=false`) so replies come back direct. The local **Ministral** model stays as an **offline fallback** — it currently wedges on this CPU node's llama.cpp warmup, so STACKIT Qwen is the live default. All STACKIT usage draws on one shared **€250/week** budget (LiteLLM `budget_duration: 7d`): once it is exhausted the gateway returns a **graceful budget error (HTTP 429)** rather than failing hard, and the pool auto-resets weekly.

WireGuard (optional, off by default). A first-class, durable in-cluster tunnel component (a UDP LoadBalancer) lets LiteLLM reach an **external** Ollama model — e.g. a Mac-Mini `qwen3:14b` on the `local-qwen` route — over a private link. Being part of the chart, it survives redeloys and node-rolls; enable it only when you want to bridge to an outside model.

5.6 Use the OS from Claude or ChatGPT (MCP)

The platform exposes itself as **governed MCP servers** you can import into Claude, ChatGPT or any MCP client. One **overarching** authenticated remote server at `/api/mcp` (Streamable-HTTP, a per-user token, role-scoped) surfaces the OS's cross-tab tools; alongside it, **per-tab** servers at `/api/mcp/<tab>` (`software`, `data`, `science`, `knowledge`, `agents`) each ship a token-minimal `CONTEXT.md`, so a

client gets just that tab's tools and just enough context. Every MCP tool delegates to the **same governed library function the UI calls**, under the caller's delegated identity – OPA policy, role gates and Langfuse audit apply unchanged, and there is no privileged path.

5.7 Memory and the build-and-toggle defaults

The full L1+L2+L3 set is sized for a large STACKIT node. To fit a 14 GB local VM, a few heavy components ship **off** by default locally (Docling, OpenSearch Dashboards, OpenMetadata, Spark, the Layer-4 Science stack, plus Terminal and Workbench). Turn any of them on from **Platform → Components** (a runtime on/off that scales 0↔1) or permanently by setting `<component>.enabled: true` and re-running `helm upgrade`. *Off* means installed but scaled to zero; *disabled* means not deployed at all.

5.8 Components at a glance

Grouped by layer – see `docs/components/<id>.md` for the full per-component guide.

LAYER	COMPONENTS
L1 – Agent core	LiteLLM (model + MCP gateway, per-key budget cap) • model-server (self-hosted Ministral 3 / Magistral 24B; two-tier STACKIT Qwen reasoning/execution on the live deploy) • mock-model (offline embeddings + fallback) • OpenSearch (retrieval) • Langfuse (tracing) • query-tool (Trino MCP) • sample & poet agents
L2 – Foundations	OPA • Docling • Haystack • Dagster • dbt • Cube • OpenMetadata
Infra	Postgres (CloudNativePG) • ClickHouse • Valkey • MinIO (object storage, PVC-backed) • Polaris (Iceberg catalog, durable relational-JDBC metastore)
L3 – Self-service	central Trino • Superset • Forgejo (sovereign git) • Argo CD • CI runner/build • OpenSearch Dashboards • Workbench • Terminal
L4 – Science	JupyterHub • MLflow • Featureform • KServe (all opt-in)
Security & platform	egress-proxy • web_fetch • WireGuard tunnel (optional) • OS UI (embedded Components console • same-origin tool proxy + Level-1 SSO • remote & per-tab MCP servers)

5.9 Full demo-login table (profile `local`)

CONSOLE	PORT-FORWARD (<code>KUBECTL -N AGENTIC-OS ...</code>)	URL	LOGIN
OS UI	<code>port-forward svc/os-ui 8080:3000</code>	<code>http://localhost:8080</code>	—
Langfuse	<code>port-forward svc/agenti-c-os-langfuse-web 3000:3000</code>	<code>http://localhost:3000</code>	<code>admin@datamasterclass.com / langfuse-local-dev-admin</code>
LiteLLM	<code>port-forward svc/agenti-c-os-litellm 4000:4000</code>	<code>http://localhost:4000/ui</code>	<code>admin / litellm-admin-local-dev</code>
Superset	<code>port-forward svc/agenti-c-os-superset 8088:8088</code>	<code>http://localhost:8088</code>	<code>admin / superset-admin-local-dev</code>
Forgejo	<code>port-forward svc/forgejo-http 3001:3000</code>	<code>http://localhost:3001</code>	<code>gitea_admin / forgejo-admin-local-dev</code>
Argo CD	<code>port-forward svc/argocd-server 8082:80</code>	<code>http://localhost:8082</code>	<code>admin / secret argocd-initial-admin-secret</code>
MinIO	<code>port-forward svc/minio 9001:9001</code>	<code>http://localhost:9001</code>	<code>agenti-c-os-local / agenti-c-os-local-secret</code>
Cube	<code>port-forward svc/cube 4001:4000</code>	<code>http://localhost:4001</code>	— (playground)
Dagster	<code>port-forward svc/agenti-c-os-dagster-webserver 3070:80</code>	<code>http://localhost:3070</code>	—
OpenMetadata*	<code>port-forward svc/openmetadata 8585:8585</code>	<code>http://localhost:8585</code>	<code>admin@open-metadata.org / admin</code>
Polaris	<code>port-forward svc/polaris 8181:8181</code>	<code>http://localhost:8181</code>	<code>root / polaris-local-dev-secret (OAuth2)</code>
OpenSearch (API)	<code>port-forward svc/opensearch 9200:9200</code>	<code>http://localhost:9200</code>	—

*Off by default locally – enable first (Platform → Components toggle, or `enabled: true` + `helm upgrade`).

5.10 Troubleshooting

- **ImagePullBackOff** on **demo-app** **right after install** is expected – it clears once the first CI run builds and bumps the image tag.
- **Out of memory / pods pending.** The slice is RAM-bound – keep the heavy components off locally or give the VM more RAM.
- **Where do I log in first?** Locally the OS UI needs no login; Langfuse (`admin@datamasterclass.com` / `langfuse-local-dev-admin`) is the default admin-style back-end console.
- **Agent answers look canned** → the self-hosted model (`model-server`) is disabled, so the offline mock model is answering. Enable `modelServer` or point LiteLLM at any model – no agent change.
- **web_fetch** **returns 403 / 502** → 403 means OPA hasn't granted the principal `web_fetch` ; 502 means the domain isn't on the egress allowlist. Both are by design.
- **No verification/invite email is sent** → no mailer is configured. The bootstrap admin **auto-verifies**, so you can start without one; to enable email, set the Microsoft Graph app-registration (`Mail.Send`) or the `SMTP_*` variables (sender `support@datamasterclass.com`).
- **Do I have to use STACKIT?** No – any Kubernetes works; the chart is portable. STACKIT is the sovereign EU default. You can build and validate everything on local `kind` with no cloud key.

5.11 Version & changelog

- **Chart 0.2.10 • app 0.2.0-alpha.11** . This build: generated `2026-07-02` from commit `410a249` .
- **This build (os-ui 0.1.4).** Every tab assistant is now **agentic** – PLAN (reasoning tier) → act/codegen (execution tier, tool-calling) → verify → **gated deploy** – not just chat; the Software build assistant genuinely scaffolds → commits → previews → requests deploy. The OS is importable into **Claude/ChatGPT as governed MCP**: one authenticated remote server at `/api/mcp` plus per-tab servers at `/api/mcp/<tab>` , each re-using the UI's governed functions. Back-end tools now embed **same-origin** through the OS UI (`/tools/<tool>`) with **Level-1 header SSO** that auto-provisions per-role accounts. The **lakehouse is durable** – MiniIO on a PVC, Polaris on a relational-JDBC (Postgres) metastore, and a `polaris-catalog-init` Job that registers the `lakehouse` catalog on deploy. On STACKIT, LiteLLM serves a **two-tier Qwen** stack (`sovereign-reasoning` = Qwen3-VL-235B, `sovereign-default` = Qwen3.6-27B; thinking disabled) under a shared **€250/week** budget cap with graceful 429, local Ministral kept as offline fallback; an optional durable **WireGuard** tunnel can bridge LiteLLM to an external Ollama model.
- **0.2.x – the OS UI v1.0:** every sidebar tab a real surface, brand-themed, light/dark, with the operational console embedded at Platform → Components; Layers 1–3 in place under the secure-by-default baseline; Science (L4) opt-in. This release's **UI rework: Home** is now the golden-path launcher only and the live cockpit modules moved to a new **Cockpit** tab (with a top-items-per-artifact board); **Strategy** is exactly three sections (Big Bets • Self Service • Foundations); **Agents** and **Software** are each one page (Software on sovereign in-cluster Forgejo – no accounts/tokens); **Monitoring** is artifact-observability only while infra health moved to **Components**; **Connections** is

external-systems only; **Platform Admin** is labelled **Admin**; terminology is **Personal + domain** throughout; the left-nav was reordered (Home, Cockpit, Strategy, Big Bets, Agents, Software, Science, Knowledge, Files, Data, Metrics, Dashboards, Connections, Marketplace, Monitoring, Governance, Settings); and first-run gained an **auto-verifying bootstrap admin** with optional email via **Microsoft Graph** `sendMail` or **SMTP**.

- **Next** – wiring the remaining Connections create/build flows from scaffold to live, broader tutorial anchors, and full per-domain spaces. Bump this section additively as the OS evolves and re-run `scripts/build-docs.sh`.

Sovereign Agentic OS – built from permissively-licensed open source for EU data residency. This guide is generated from the repository; to update it, edit `docs/Sovereign-Agentic-OS-Guide.md` and run `scripts/build-docs.sh`.